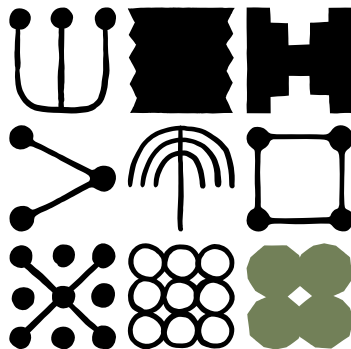


详解 AI Agent 评测

Demystifying evals for AI agents

Anthropic · 2026-01-09 · 原文 / Source

完整译文与英文逐段对照。依据你提供的网页存档翻译，保留原文的观点、例子与限定条件。图表（含图注）、代码及其中的文字维持原文；网页导航、订阅表单和站点页脚不属于文章正文。中文为学习用非官方译文。



让 Agent 发挥作用的那些能力，也让它们难以评测。能够适用于不同部署场景的评测策略，会组合使用多种技术，以匹配被评测系统的复杂程度。

引言

好的评测能让团队更有信心地交付 AI Agent。缺少评测，团队很容易陷入被动应对的循环：问题直到生产环境中才被发现，而修复一个故障又会引发其他故障。评测能够在问题和行为变化影响用户之前，就让它们显现出来；它的价值还会在 Agent 的整个生命周期中不断累积。

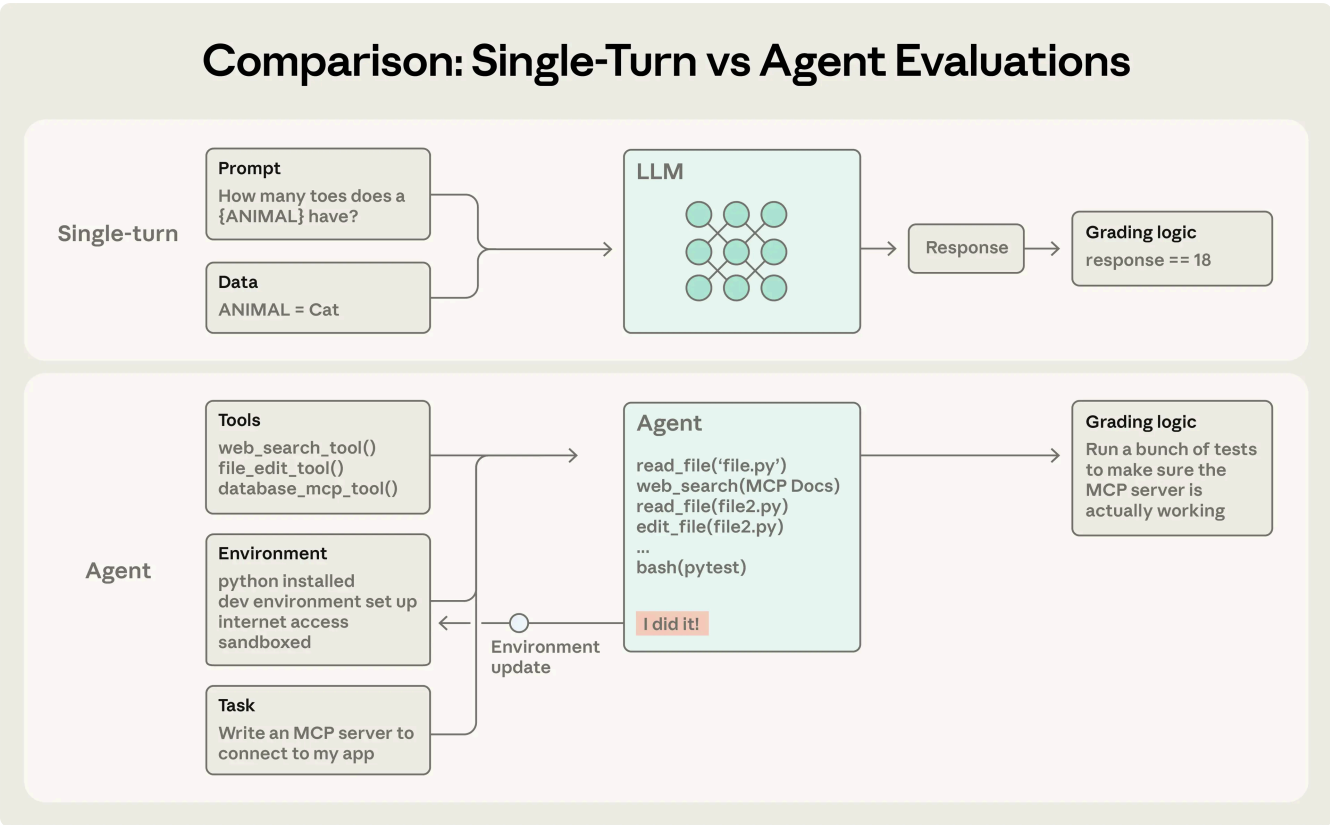
正如我们在《构建高效的 Agent》中所述，Agent 会跨越多个轮次运行：调用工具、修改状态，并根据中间结果调整行动。让 AI Agent 发挥作用的自主性、智能与灵活性，也恰恰让它们更难评测。

通过内部实践，以及与处于 Agent 开发前沿的客户合作，我们学会了如何为 Agent 设计更严谨、更有用的评测。下面介绍的是在真实部署中，适用于多种 Agent 架构和使用场景的方法。

评测的结构

评测（evaluation，简称“eval”）是对 AI 系统进行的一种测试：向 AI 提供输入，再对其输出应用评分逻辑，以衡量是否成功。本文重点讨论自动化评测，即在开发过程中无需真实用户参与就能运行的评测。

单轮评测很直接：一条提示、一个回复，以及评分逻辑。对于早期的大语言模型，单轮、非 Agent 式评测是主要的评测方法。随着 AI 能力不断增强，多轮评测变得越来越普遍。



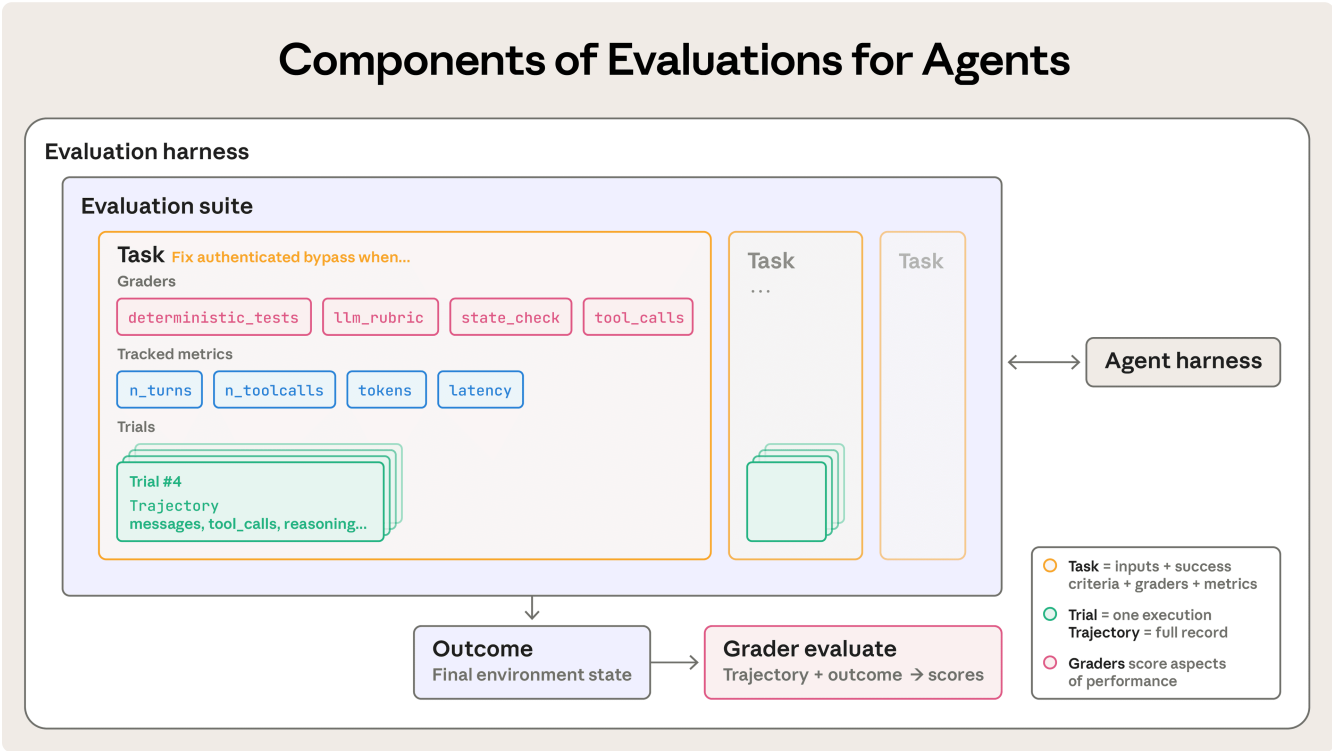
In a simple eval, an agent processes a prompt, and a grader checks if the output matches expectations. For a more complex multi-turn eval, a coding agent receives tools, a task (building an MCP server in this case), and an environment, executes an "agent loop" (tool calls and reasoning), and updates the environment with the implementation. Grading then uses unit tests to verify the working MCP server.

Agent 评测则更加复杂。Agent 在多个轮次中使用工具，改变环境中的状态，并在过程中不断调整——这意味着错误可能传播并累积。前沿模型还可能找到超出静态评测设定范围的创造性解法。例如，Opus 4.5 在解决 τ 2-bench 中一道航班预订问题时，发现了业务规则中的一个漏洞。按评测原有的写法，它“未通过”测试，但实际上为用户找到了更好的解决方案。

构建 Agent 评测时，我们采用以下定义：

- **任务** (task, 也称**问题**或**测试用例**) 是一次独立测试，具有明确的输入和成功标准。
- 对任务的每一次尝试称为**单次试验** (trial)。由于模型在不同运行中的输出会有变化，我们会执行多次试验，以获得更稳定的结果。
- **评分器** (grader) 是对 Agent 表现的某个方面进行评分的逻辑。一个任务可以有多个评分器，每个评分器又可以包含多个断言 (有时称为**检查项**)。
- **完整记录** (transcript, 也称**执行轨迹**, 即 trace 或 trajectory) 是单次试验的全部记录，包括输出、工具调用、推理、中间结果及其他所有交互。对于 Anthropic API, 它指评测运行结束时完整的 messages 数组，包含评测期间所有 API 调用及其返回的全部响应。
- **实际结果** (outcome) 是单次试验结束时环境的最终状态。航班预订 Agent 可能在完整记录的结尾说“您的航班已预订”，但实际结果要看环境中的 SQL 数据库里是否真的存在一条预订记录。

- **评测运行框架**（evaluation harness）是端到端执行评测的基础设施。它提供指令和工具，并发运行任务，记录所有步骤，为输出评分，并汇总结果。
- **Agent 运行框架**（agent harness，也称 **scaffold**）是使模型能够作为 Agent 行动的系统：它处理输入、编排工具调用并返回结果。当我们评测“一个 Agent”时，评测的是运行框架与模型协同工作的表现。例如，Claude Code 是一个灵活的 Agent 运行框架，我们通过 Agent SDK 使用其核心原语，构建了我们的长时间运行 Agent 的运行框架。
- **评测套件**（evaluation suite）是为衡量特定能力或行为而设计的一组任务。套件中的任务通常具有共同的大方向。例如，客户支持评测套件可能会测试退款、取消和升级处理。



Components of evaluations for agents.

为什么要建设评测？

刚开始构建 Agent 时，团队通过结合手动测试、自己使用自己的产品以及直觉判断，就能取得令人意外的进展。更严格的评测甚至可能被视为拖慢交付速度的额外负担。但在早期原型阶段之后，一旦 Agent 投入生产并开始扩大规模，缺少评测的开发方式就开始难以为继。

转折点往往出现在用户反馈“改完以后 Agent 感觉变差了”的时候。团队如同“盲飞”，除了猜测和反复尝试，无法验证这一反馈。没有评测，调试就只能被动进行：等待用户投诉、手动复现、修复缺陷，然后希望其他地方没有退步。团队无法区分真正的性能退化与噪声，无法在上线前自动用数百种场景测试改动，也无法衡量改进的幅度。

我们曾多次见到这样的发展过程。例如，Claude Code 最初依靠 Anthropic 员工和外部用户的反馈快速迭代。后来，我们加入了评测：先覆盖简洁程度和文件编辑等较窄的领域，再扩展到过度设计等更复杂的行为。这些评测帮助我们发现问题、指导改进，并为研究与产品团队的合作确定重点。结合生产监控、A/B 测试、用户研究等方法，评测为我们提供了信号，使 Claude Code 在规模扩大时仍能持续改进。

在 Agent 生命周期的任何阶段，编写评测都有价值。早期，评测迫使产品团队明确 Agent 的成功意味着什么；后期，评测则帮助维持一致的质量标准。

Descript 的 Agent 帮助用户编辑视频，因此他们围绕成功编辑流程的三个维度构建了评测：不要破坏现有内容、完成我的要求、把事情做好。他们从人工评分逐步转向 LLM 评分器，评分标准由产品团队制定，并定期通过人工进行校准；如今，他们定期运行两套独立的评测套件，分别用于质量基准测试和回归测试。Bolt AI 团队开始建设评测的时间更晚，当时他们已经拥有一个被广泛使用的 Agent。他们用 3 个月建成了一套评测系统：运行 Agent，通过静态分析为输出评分，使用浏览器 Agent 测试应用，并借助 LLM 裁判评价指令遵循等行为。

有些团队在开发之初就创建评测；另一些则等到系统已经达到一定规模、缺少评测成为改进 Agent 的瓶颈时，才补上评测。评测在 Agent 开发初期尤其有用，因为它能把预期行为明确地编码下来。两位工程师阅读同一份初始需求说明，对 AI 应该如何处理边界情况，可能会得出不同理解。评测套件能够消除这种歧义。无论何时创建，评测都有助于加快开发。

评测也会影响你采用新模型的速度。当更强大的模型发布时，没有评测的团队需要花数周测试；而拥有评测的竞争对手能够快速确定模型的优势、调整提示词，并在几天内完成升级。

有了评测，基线和回归测试也就随之而来：你可以在一组固定任务上跟踪延迟、token 用量、单个任务成本和错误率。评测还可以成为产品与研究团队之间信息传递效率最高的沟通渠道，为研究人员定义可优化的指标。显然，评测的广泛益处远不止跟踪退化与改进。由于成本在一开始就清晰可见，而收益要到后来逐渐累积，人们很容易忽视它不断叠加的价值。

如何评测 AI Agent

目前，我们看到几类常见 Agent 正在大规模部署，包括编程 Agent、研究 Agent、计算机操作 Agent 和对话 Agent。每一类都可能部署在许多不同的行业，但可以用相似的技术进行评测。你不必从零发明评测方法。下面将介绍针对几类 Agent 的成熟技术。可以把这些方法作为基础，再扩展到你所在的领域。

Agent 评分器的类型

Agent 评测通常结合三类评分器：基于代码的、基于模型的，以及人工评分。每个评分器都对完整记录或实际结果中的某一部分进行评价。根据任务选择合适的评分器，是有效评测设计的重要组成部分。

基于代码的评分器

Methods	Strengths	Weaknesses
• String match checks (exact, regex, fuzzy, etc.) • Binary tests (fail-to-pass, pass-to-pass) • Static analysis (lint, type, security) • Outcome verification • Tool calls verification (tools used, parameters) • Transcript analysis (turns taken, token usage)	• Fast • Cheap • Objective • Reproducible • Easy to debug • Verify specific conditions	• Brittle to valid variations that don't match expected patterns exactly • Lacking in nuance • Limited for evaluating some more subjective tasks

基于模型的评分器

Methods	Strengths	Weaknesses
• Rubric-based scoring • Natural language assertions • Pairwise comparison • Reference-based evaluation • Multi-judge consensus	• Flexible • Scalable • Captures nuance • Handles open-ended tasks • Handles freeform output	• Non-deterministic • More expensive than code • Requires calibration with human graders for accuracy

人工评分

Methods	Strengths	Weaknesses
• SME review • Crowdsourced judgment • Spot-check sampling • A/B testing • Inter-annotator agreement	• Gold standard quality • Matches expert user judgment • Used to calibrate model-based graders	• Expensive • Slow • Often requires access to human experts at scale

每个任务可以采用加权评分（各评分器的得分合并后必须达到阈值）、二元评分（所有评分器都必须判定通过），也可以混合使用两种方式。

能力评测与回归评测

能力评测，也称“质量”评测，回答的是：“这个 Agent 能把什么事情做好？”它们的初始通过率应当较低，重点针对 Agent 难以完成的任务，为团队提供一个逐步提升的目标。

回归评测回答的是：“Agent 是否仍然能处理它过去能处理的所有任务？”这类评测的通过率应接近 100%。它们用于防止能力倒退，因为分数下降意味着某处出现了问题，需要改进。团队在能力评测上逐步提升表现时，也必须运行回归评测，以确保改动没有在其他地方造成问题。

Agent 上线并经过优化后，通过率较高的能力评测就可以“毕业”，成为持续运行的回归套件，用于捕捉任何偏移。那些曾经衡量“我们究竟能不能做到”的任务，此时衡量的就变成了“我们是否仍能可靠地做到”。

评测编程 Agent

编程 Agent编写、测试和调试代码，像人类开发者一样浏览代码库并运行命令。对现代编程 Agent 的有效评测，通常依赖定义清晰的任务、稳定的测试环境，以及针对生成代码的充分测试。

确定性评分器很适合编程 Agent，因为软件通常比较容易评价：代码能否运行，测试是否通过？两个广泛使用的编程 Agent 基准——SWE-bench Verified 和 Terminal-Bench——都采用这种方法。SWE-bench Verified 向 Agent 提供热门 Python 仓库中的 GitHub issue，并通过运行测试套件为解决方案评分；只有修复原本失败的测试，同时不破坏已有测试，解决方案才算通过。短短一年内，LLM 在这项评测上的成绩就从 40% 提高到超过 80%。Terminal-Bench 走的是另一条路线：它测试端到端的技术任务，例如从源码构建 Linux 内核，或者训练一个机器学习模型。

有了一组通过 / 失败测试，用于验证编程任务的关键实际结果之后，通常也值得对完整记录进行评分。例如，基于启发式的代码质量规则，可以从是否通过测试之外的角度评价生成代码；而配有明确评分细则的模型评分器，则可以评估 Agent 如何调用工具、如何与用户互动等行为。

示例：编程 Agent 的假设性评测

假设有一项编程任务，要求 Agent 修复一个身份认证绕过漏洞。如下面的示例 YAML 文件所示，可以同时使用评分器和指标来评测这个 Agent。

```
task:
  id: "fix-auth-bypass_1"
  desc: "Fix authentication bypass when password field is empty and ..."
  graders:
    - type: deterministic_tests
      required: [test_empty_pw_rejected.py, test_null_pw_rejected.py]
    - type: llm_rubric
      rubric: prompts/code_quality.md
    - type: static_analysis
      commands: [ruff, mypy, bandit]
    - type: state_check
      expect:
        security_logs: {event_type: "auth_blocked"}
    - type: tool_calls
      required:
        - {tool: read_file, params: {path: "src/auth/*"}}
        - {tool: edit_file}
        - {tool: run_tests}
  tracked_metrics:
    - type: transcript
      metrics:
        - n_turns
        - n_toolcalls
        - n_total_tokens
    - type: latency
      metrics:
        - time_to_first_token
        - output_tokens_per_sec
        - time_to_last_token
```

请注意，为了说明可用的方法，这个示例展示了各类评分器。在实际应用中，编程评测通常依靠单元测试验证正确性，并通过面向 LLM 的评分细则评价整体代码质量；只有在需要时才加入额外的评分器和指标。

评测对话 Agent

对话 Agent在客服、销售或辅导等领域与用户互动。与传统聊天机器人不同，它们会维护状态、使用工具，并在对话过程中采取行动。虽然编程 Agent 和研究 Agent 也可能与用户进行多轮互动，但对话 Agent 有一个独特的挑战：交互本身的质量，就是评测对象的一部分。对话 Agent 的有效评测通常依靠可验证的环境最终状态，以及兼顾任务完成度和交互质量的评分细则。与大多数其他评测不同，这类评测往往需要第二个 LLM 来模拟用户。我们在对齐审计 Agent 中采用了这种方法，通过持续的对抗性对话对模型进行压力测试。

对话 Agent 的成功可以有多个维度：工单是否解决（状态检查）、是否在少于 10 轮内完成（完整记录约束）、语气是否恰当（LLM 评分细则）？ τ -Bench 及其后继版本 τ 2-Bench 就是两个采用多维度评价的基准。它们模拟零售客服、航班预订等领域的多轮交互：由一个模型扮演特定用户角色，Agent 则在贴近真实的场景中处理任务。

示例：对话 Agent 的假设性评测

假设有一项客服任务，要求 Agent 为一位心情不满的客户处理退款。

```
graders:
- type: llm_rubric
  rubric: prompts/support_quality.md
  assertions:
    - "Agent showed empathy for customer's frustration"
    - "Resolution was clearly explained"
    - "Agent's response grounded in fetch_policy tool results"
- type: state_check
  expect:
    tickets: {status: resolved}
    refunds: {status: processed}
- type: tool_calls
  required:
    - {tool: verify_identity}
    - {tool: process_refund, params: {amount: "<=100"}}
    - {tool: send_confirmation}
- type: transcript
  max_turns: 10
tracked_metrics:
- type: transcript
  metrics:
    - n_turns
    - n_toolcalls
    - n_total_tokens
- type: latency
  metrics:
    - time_to_first_token
    - output_tokens_per_sec
    - time_to_last_token
```

与编程 Agent 的示例一样，这项任务为了说明方法而展示了多种评分器。在实际应用中，对话 Agent 评测通常使用基于模型的评分器，同时评价沟通质量和目标完成情况，因为许多任务——例如回答问题——可能多个“正确”的解决方案。

评测研究 Agent

研究 Agent 收集、综合并分析信息，然后给出答案或报告等产出。编程 Agent 可以通过单元测试获得通过 / 失败的二元信号，而研究质量只能结合具体任务来判断。什么算“全面”“来源充分”，甚至什么算“正确”，都取决于具体情境：市场扫描、并购尽职调查和科学报告，各自需要不同的标准。

研究评测面临独特的挑战：专家可能对综合分析是否全面存在分歧；参考内容不断变化，标准答案也随之改变；而更长、更开放的输出也带来了更多出错空间。例如，BrowseComp 这样的基准会测试 AI Agent 是否能在开放网络上大海捞针——它的问题被设计为易于验证、却难以解答。

构建研究 Agent 评测的一种策略，是组合使用多类评分器。依据性检查验证论断是否得到检索来源的支持；覆盖度检查定义一份好答案必须包含哪些关键事实；来源质量检查确认参考来源具有权威性，而不只是最先检索到的结果。对于有客观正确答案的任务（“X 公司第三季度的营收是多少？”），可以采用精确匹配。LLM 能标记缺少依据的论断和覆盖遗漏，也能检查开放式综合分析的连贯性与完整性。

由于研究质量具有主观性，要有效评测这类 Agent，就应经常以人类专家的判断为参照，校准基于 LLM 的评分细则。

计算机操作 Agent

计算机操作 Agent通过与人类相同的界面来使用软件——截图、鼠标点击、键盘输入和滚动——而不是通过 API 或代码执行。它们能使用任何具有图形用户界面（GUI）的应用，从设计工具到传统企业软件都包括在内。评测需要让 Agent 在真实环境或沙箱环境中运行，使其能够使用软件应用，并检查它是否实现了预期结果。例如，WebArena 测试基于浏览器的任务，通过 URL 和页面状态检查验证 Agent 的导航是否正确；对于修改数据的任务，还会验证后端状态，确认订单确实已经提交，而不仅仅是出现了确认页面。OSWorld 将这一方法扩展到对整个操作系统的控制，利用评测脚本在任务完成后检查各种产物：文件系统状态、应用配置、数据库内容，以及 UI 元素属性。

浏览器操作 Agent 需要在 token 效率与延迟之间取得平衡。基于 DOM 的交互执行速度快，但会消耗大量 token；基于截图的交互更慢，却更节省 token。例如，让 Claude 总结维基百科内容时，从 DOM 中提取文本更高效；在亚马逊上寻找一个新的笔记本电脑保护套时，截图更高效，因为提取整个 DOM 会消耗大量 token。在 Claude for Chrome 产品中，我们开发了评测，用来检查 Agent 是否为每种情境选择了正确的工具。这使我们能够更快、更准确地完成浏览器任务。

如何理解 Agent 评测中的非确定性

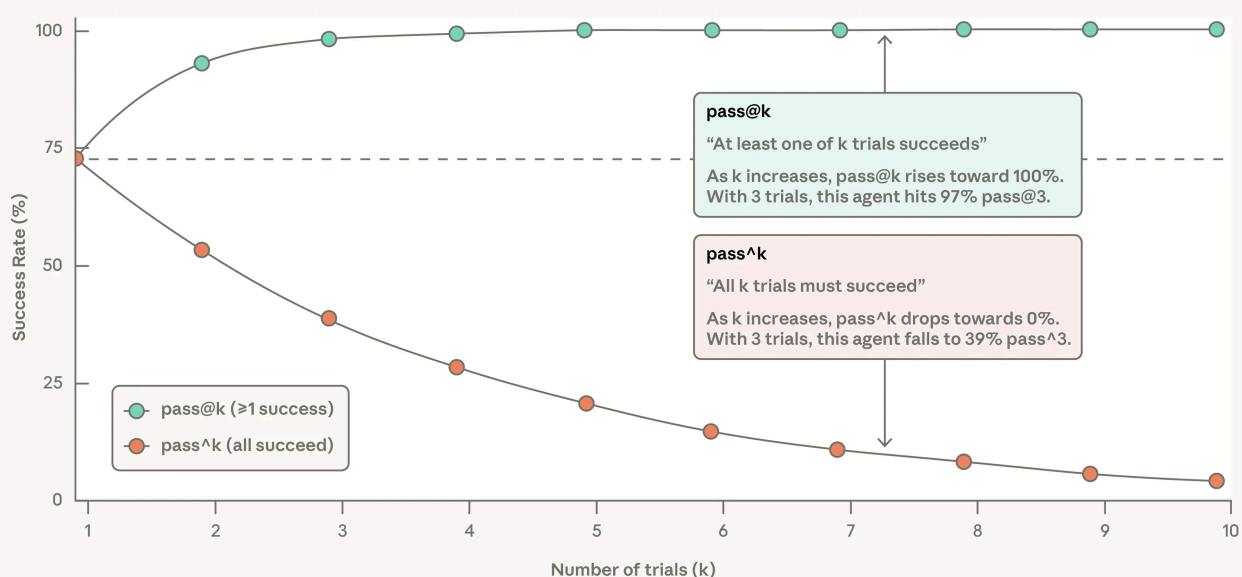
无论哪一类 Agent，不同运行之间的行为都会变化，因此评测结果比初看起来更难解读。每个任务都有自己的成功率——某个任务可能是 90%，另一个则是 50%——而在一次评测运行中通过的任务，下一次可能会失败。有时，我们想衡量的是：Agent 在一项任务上成功的频率究竟是多少，也就是成功试验占全部试验的比例。

有两个指标可以帮助体现这种差别：

pass@k 衡量 Agent 在 k 次尝试中至少找到一个正确解法的概率。随着 k 增大，pass@k 分数会上升：更多次“射门”意味着至少成功 1 次的可能性更高。pass@1 为 50%，意味着模型在评测中有一半任务第一次尝试就成功。在编程场景中，我们往往最关心 Agent 能否一次就找到解决方案，即 pass@1。在其他场景中，只要有一个方案可行，提出多个方案也可以接受。

pass^k 衡量 k 次试验全部成功的概率。随着 k 增大，pass^k 会下降，因为要求更多次试验都保持成功，门槛也就更高。如果 Agent 的单次试验成功率为 75%，执行 3 次试验时，三次全部通过的概率就是 $(0.75)^3 \approx 42\%$ 。对于面向客户的 Agent，这个指标尤为重要，因为用户希望每一次都获得可靠的表现。

The Difference Between Pass@k and Pass^k



pass@k and pass^k diverge as trials increase. At k=1, they're identical (both equal the per-trial success rate). By k=10, they tell opposite stories: pass@k approaches 100% while pass^k falls to 0%.

这两个指标都有用，采用哪一个取决于产品需求：对于只要成功一次就有价值的工具，使用 pass@k；对于一致性至关重要的 Agent，使用 pass^k。

从零到一：建设优质 Agent 评测的路线图

本节介绍我们经过实践检验的建议，帮助你从没有评测，走向拥有可信赖的评测。可以把它视为评测驱动的 Agent 开发路线图：尽早定义成功、明确衡量成功，并持续迭代。

为初始评测数据集收集任务

第 0 步：尽早开始

我们看到一些团队迟迟不建设评测，因为他们认为需要数百个任务。实际上，从真实失败中提取 20—50 个简单任务，就是很好的起点。毕竟，在 Agent 开发早期，对系统的每次改动通常都会带来清晰、明显的影响；效应量较大，意味着较小的样本量就足够。更成熟的 Agent 可能需要规模更大、难度更高的评测，才能检测到较小的效果变化，但初期最好采用二八原则。拖得越久，评测就越难建设。早期，产品需求可以自然地转化为测试用例；等得太久，你就只能从一个正在运行的系统中反向推导成功标准。

第 1 步：从你已经在手动测试的内容入手

从开发过程中执行的手动检查开始：每次发布前都会验证的行为，以及最终用户经常尝试的任务。如果产品已经上线，就查看缺陷跟踪系统和客服工单队列。将用户报告的失败转化为测试用例，能确保评测套件反映真实使用情况；按对用户的影响排序，则能帮助你把精力投入最有价值的地方。

第 2 步：编写没有歧义的任务，并提供参考解法

把任务质量做好，比看起来更难。一个好任务，应当让两位领域专家各自独立判断时，都能得出相同的通过 / 失败结论。他们自己能完成这个任务吗？如果不能，任务就需要改进。任务说明中的歧义会变成指标中的噪声。基于模型的评分器也一样：模糊的评分细则会产生不一致的判断。

每个任务都应该能够被正确遵循指令的 Agent 完成。其中的问题有时很细微。例如，对 Terminal-Bench 的审查发现：如果任务要求 Agent 编写一个脚本，却没有指定文件路径，而测试又假定脚本位于某个特定路径，那么 Agent 可能并没有做错，却仍然失败。评分器检查的所有内容，都应当能从任务描述中明确得知；不应因说明有歧义而判定 Agent 失败。对于前沿模型，如果多次试验的通过率都是 0%（即 pass@100 为 0%），往往意味着任务本身有问题，而不是 Agent 能力不足；此时应重新仔细检查任务说明和评分器。为每项任务创建一个参考解法很有用，即一个已知可行、能通过全部评分器的输出。这能证明任务确实可解，并验证评分器配置正确。

第 3 步：构建均衡的问题集

既要测试某种行为应该发生的情况，也要测试它不应该发生的情况。单边评测会导致单边优化。例如，如果你只测试 Agent 在该搜索时是否搜索，最终可能得到一个几乎事事都搜索的 Agent。应尽量避免类别不均衡的评测。我们在为 Claude.ai 的网页搜索构建评测时，亲身体会到了这一点。挑战在于：既要防止模型在不该搜索时搜索，又要保留它在适当情况下开展深入研究的能力。团队构建了覆盖两个方向的评测：一类是模型应当搜索的查询，例如查询天气；另一类是模型应根据已有知识回答的查询，例如“谁创立了苹果公司？”。在触发不足（该搜索却不搜索）和过度触发（不该搜索却搜索）之间找到合适的平衡很难，需要对提示词和评测进行多轮改进。随着更多问题样例出现，我们还会继续补充评测，以扩大覆盖范围。

设计评测运行框架与评分器

第 4 步：构建稳健的评测运行框架，并提供稳定环境

评测中的 Agent 应当与生产环境中的 Agent 以大致相同的方式运行，而且环境本身不能引入额外噪声，这一点至关重要。每次试验都应从干净的环境开始，做到彼此“隔离”。不同运行之间不必要的共享状态，例如残留文件、缓存数据、资源耗尽，可能引发相关性失败；这些失败来自基础设施的不稳定，而非 Agent 本身的表现。共享状态也可能人为抬高成绩。例如，在一些内部评测中，我们发现 Claude 通过查看先前试验留下的 git 历史，在某些任务上获得了不公平的优势。如果多个不同试验都因同一种环境限制而失败，例如 CPU 内存有限，那么它们就不是独立试验，因为受到了同一个因素的影响；此时，用这些评测结果衡量 Agent 表现就不可靠。

第 5 步：认真设计评分器

如前所述，优秀的评测设计，需要为 Agent 和任务选择最合适的评分器。我们建议：能用确定性评分器时优先使用；必要时，或需要额外灵活性时，使用 LLM 评分器；再有选择地采用人工评分，进行补充验证。

人们常会本能地检查 Agent 是否遵循了非常具体的步骤，例如是否按正确顺序调用了一系列工具。我们发现这种方法过于僵化，会让测试变得十分脆弱，因为 Agent 经常能找到评测设计者没有预料到的有效方法。为了避免不必要地惩罚创造性，通常更好的做法是评价 Agent 产出了什么，而不是它走了哪条路径。

对于包含多个组成部分的任务，应当设置部分得分。一个正确识别了问题、验证了客户身份，但未能完成退款的客服 Agent，显然比一开始就失败的 Agent 更好。结果应当体现这种成功程度的连续变化。

模型评分往往需要认真迭代，才能验证其准确性。充当裁判的 LLM（LLM-as-judge）评分器应与人类专家密切校准，以确信人工评分和模型评分之间的偏差很小。为避免幻觉，要给 LLM 留出退路，例如指示它在信息不足时返回“Unknown”。为任务的每个维度制定清晰、结构化的评分细则，也会有所帮助；然后，让独立的 LLM 裁判分别评价各个维度，而不是由一个裁判评价所有维度。系统稳定可靠后，只需偶尔进行人工复核即可。

有些评测存在隐蔽的失效方式，即便 Agent 表现不错，也会得到低分：评分缺陷、Agent 运行框架的限制或歧义，使 Agent 无法完成任务。即使经验丰富的团队也可能漏掉这些问题。例如，Opus 4.5 最初在 CORE-Bench 上只得到 42%，后来一位 Anthropic 研究人员发现了多处问题：评分过于死板，在期望值为“96.124991...”时，会把“96.12”判为错误；任务说明有歧义；还有无法精确复现的随机性任务。修复缺陷并采用限制更少的运行框架之后，Opus 4.5 的得分跃升至 95%。类似地，METR 发现，其时间跨度基准中有几项任务配置有误：任务要求 Agent 优化到指定分数阈值，但评分却要求超过该阈值。这使 Claude 等遵循指令的模型受到了惩罚，反而是忽略既定目标的模型得分更高。认真复查任务和评分器，有助于避免这些问题。

要让评分器能够抵御绕过或投机取巧。Agent 不应轻易就能在评测中“作弊”。任务和评分器的设计应确保：要通过评测，就必须真正解决问题，而不能利用非预期的漏洞。

长期维护和使用评测

第 6 步：检查完整记录

如果不阅读大量试验的完整记录和评分，你就无法知道评分器是否运行良好。在 Anthropic，我们投入资源开发了查看评测完整记录的工具，并定期花时间阅读这些记录。任务失败时，完整记录能告诉你：到底是 Agent 真正犯了错，还是评分器拒绝了一个有效的解决方案。它通常还会揭示 Agent 和评测行为中的关键细节。

失败的判定应当让人觉得公平：Agent 哪里做错了、为什么错，都应清晰可见。当分数不再提高时，我们需要确信原因在于 Agent 的表现，而不是评测本身。阅读完整记录，正是验证评测是否在衡量真正重要的东西的方法，也是 Agent 开发的一项关键技能。

第 7 步：留意能力评测是否趋于饱和

通过率为 100% 的评测能够跟踪回归，但不能提供改进信号。当 Agent 通过了所有可解任务，不再有提升空间时，就发生了**评测饱和**。例如，SWE-Bench Verified 的分数在今年年初还是 30%，而现在前沿模型已超过 80%，正逐渐接近饱和。随着评测接近饱和，由于只剩最难的任务，进展也会放缓。这可能让结果具有误导性：很大的能力提升，在分数上却只体现为很小的增长。例如，代码审查初创公司 Qodo 最初并未觉得 Opus 4.5 的表现有多突出，因为他们的单次作答式编程评测没有捕捉到它在更长、更复杂任务上的进步。为此，他们开发了一套新的 Agent 式评测框架，从而更清楚地看到了能力进展。

我们的原则是：在有人深入查看评测细节、阅读一些完整记录之前，不会仅凭分数表面就接受评测结论。如果评分不公平、任务含糊不清、有效解法受到惩罚，或者运行框架限制了模型，就应该修订评测。

第 8 步：通过开放贡献和持续维护，让评测套件长期保持有效

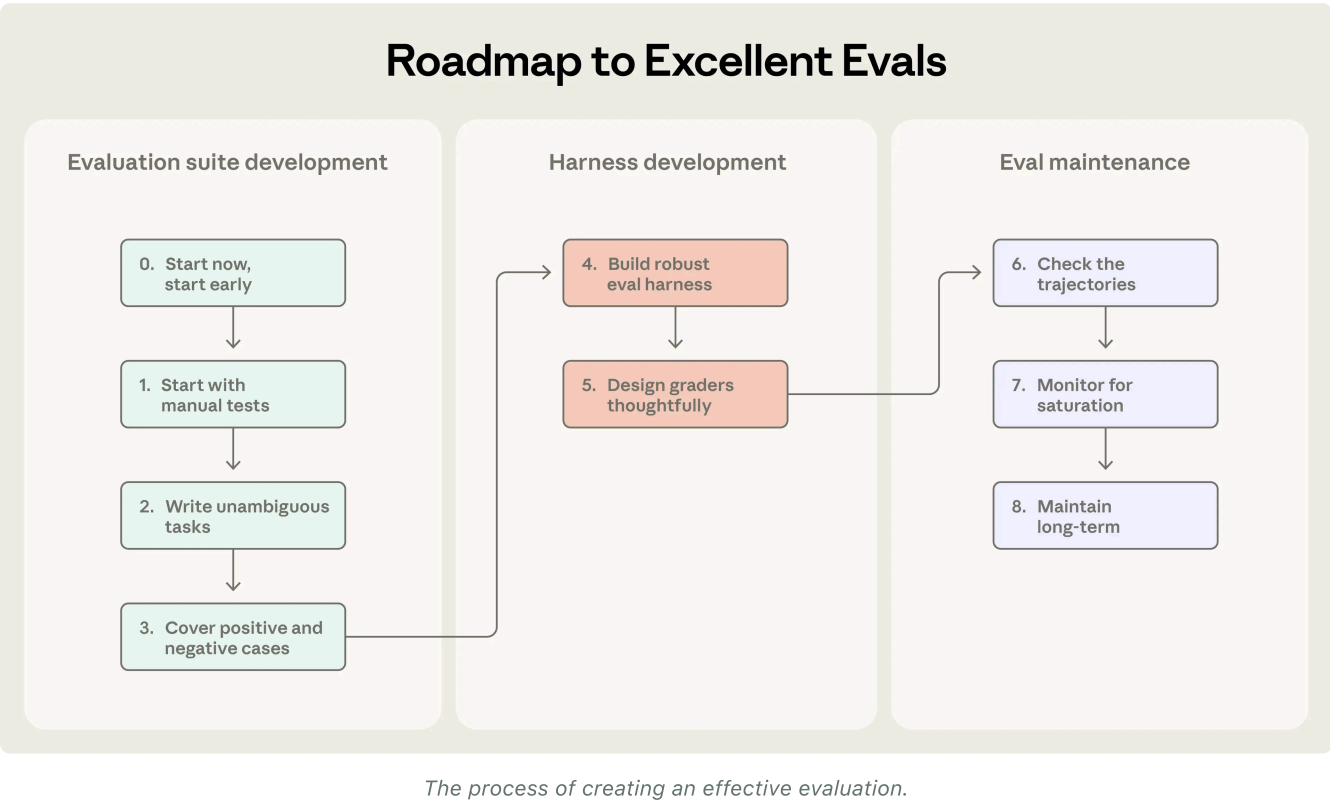
评测套件是一项不断演进的成果，需要持续投入精力，并明确责任归属，才能一直发挥作用。

在 Anthropic，我们尝试过多种维护评测的方法。事实证明，最有效的方式是设立专门的评测团队，负责核心基础设施；与此同时，由领域专家和产品团队贡献大多数评测任务，并自行运行评测。

对于 AI 产品团队，负责和迭代评测应当像维护单元测试一样日常。团队可能在一些 AI 功能上浪费数周：它们在早期测试中“能用”，却无法满那些没有明说的预期，而设计良好的评测原本可以及早揭示这些预期。定义评测任务，是检验产品需求是否已经具体到足以开始开发的最佳方法之一。

我们建议采用评测驱动开发：在 Agent 尚不能实现计划中的能力之前，就先构建评测来定义这些能力，然后持续迭代，直到 Agent 表现良好。在内部，我们经常构建一些当下表现只是“够用”的功能，同时押注模型几个月后能实现的能力。初始通过率较低的能力评测，能让这些押注清晰可见。当新模型发布时，运行评测套件，就能迅速看出哪些押注得到了回报。

最接近产品需求和用户的人，最有条件定义成功。凭借当前模型的能力，产品经理、客户成功经理或销售人员都可以使用 Claude Code，通过 PR 贡献一个评测任务——让他们去做！更好的做法是，主动为他们提供支持。

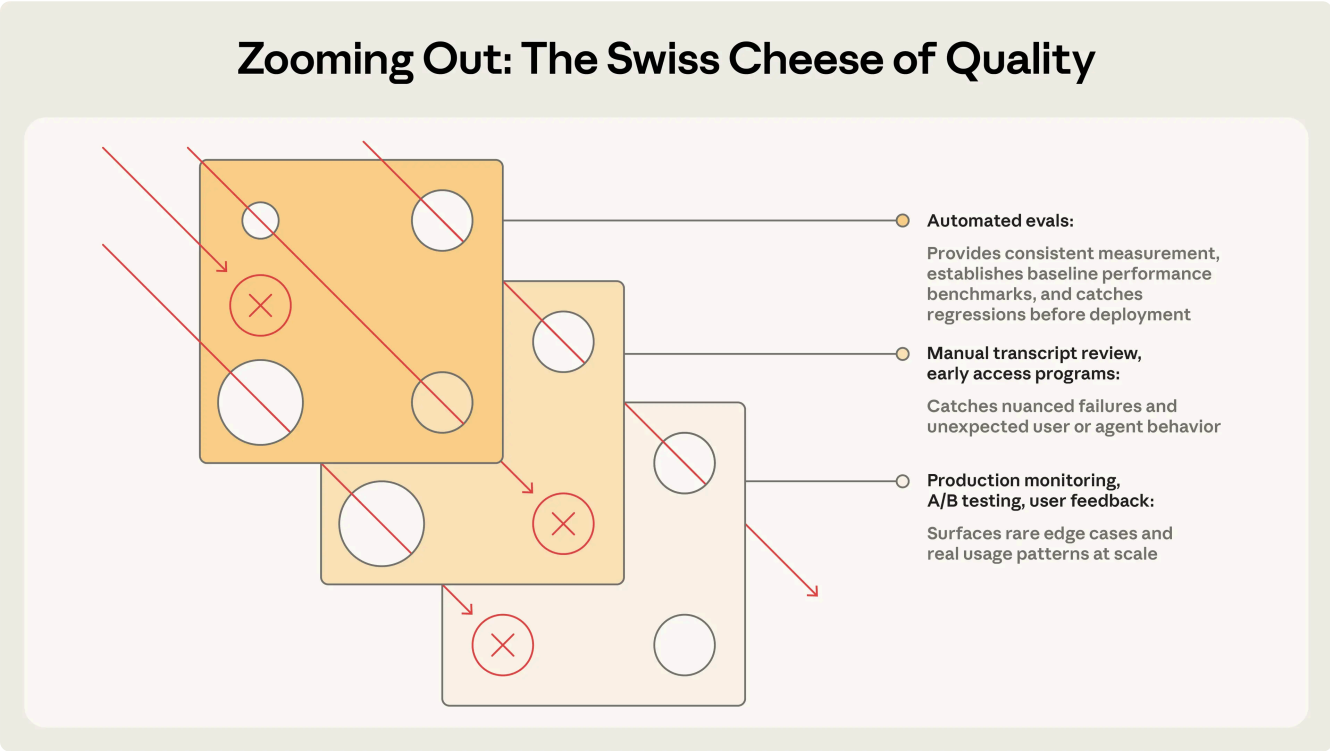


了解 AI Agent 表现的各种方法概览

Method	Pros	Cons
Automated evals <i>Running tests programmatically without real users</i>	• Faster iteration • Fully reproducible • No user impact • Can run on every commit • Tests scenarios at scale without requiring a prod deployment	• Requires more up-front investment to build • Requires ongoing maintenance as product and model evolves to avoid drift • Can create false confidence if it doesn't match real usage patterns
Production monitoring <i>Tracking metrics and errors in live systems</i>	• Reveals real user behavior at scale • Catches issues that synthetic evals miss • Provides ground truth on how agents actually perform	• Reactive; problems reach users before you know about them • Signals can be noisy • Requires investment in instrumentation • Lacks ground truth for grading
A/B testing <i>Comparing variants with real user traffic</i>	• Measures actual user outcomes (retention, task completion) • Controls for confounds • Scalable and systematic	• Slow; days or weeks to reach significance and requires sufficient traffic • Only tests changes you deploy • Less signal on the underlying "why" for changes in metrics without being able to thoroughly review the transcripts
User feedback <i>Explicit signals like thumbs-down or bug reports</i>	• Surfaces problems you didn't anticipate • Comes with real examples from actual human users • The feedback often correlates with product goals	• Sparse and self-selected • Skews toward severe issues • Users rarely explain <i>why</i> something failed • Not automated • Relying primarily on users to catch issues can have negative user impact
Manual transcript review <i>Humans reading through agent conversations</i>	• Builds intuition for failure modes • Catches subtle quality issues automated checks miss • Helps calibrate what "good" looks like and grasp details	• Time-intensive • Doesn't scale • Coverage is inconsistent • Reviewer fatigue or different reviewers can affect the signal quality • Typically only gives qualitative signal rather than clear quantitative grading

Method	Pros	Cons
Systematic human studies <i>Structured grading of agent outputs by trained raters</i>	• Gold-standard quality judgements from multiple human raters • Handles subjective or ambiguous tasks • Provides signal for improving model-based graders	• Relatively expensive and slow turnaround • Hard to run frequently • Inter-rater disagreement requires reconciliation • Complex domains (legal, finance, healthcare) require human experts to conduct studies

这些方法对应 Agent 开发的不同阶段。自动化评测在上线前和 CI/CD 中尤其有用：每次修改 Agent 或升级模型时都运行，作为防范质量问题的第一道防线。生产监控在上线后开始发挥作用，用于检测分布漂移，以及事先没有预料到的真实世界故障。有了足够流量后，可以用 A/B 测试验证重要改动。用户反馈和完整记录审阅则是持续进行的实践，用于填补空白：不断分流处理反馈，每周抽样阅读完整记录，并根据需要深入调查。系统性的人类参与研究，则留给校准 LLM 评分器，或评价以人类共识为参照标准的主观性输出。



Like the Swiss Cheese Model from safety engineering, no single evaluation layer catches every issue. With multiple methods combined, failures that slip through one layer are caught by another.

最有效的团队会结合这些方法：用自动化评测实现快速迭代，用生产监控了解实际表现，再通过定期人工复核进行校准。

结语

没有评测的团队，会困在被动应对的循环中：修复一个故障，又引发另一个故障，无法区分真正的退化与噪声。尽早投入评测的团队则会看到相反的结果：失败转化为测试用例，测试用例防止回归，指标取代猜测，开发因此加快。评测为整个团队提供了清晰的提升目标，让“Agent 感觉变差了”成为可以采取行动的问题。它的价值会不断累积，但前提是你把评测视为核心组成部分，而不是事后补上的东西。

不同类型的 Agent 所适用的具体方法各不相同，但本文介绍的基本原则是一致的。尽早开始，不要等待完美的评测套件。从你观察到的失败中提取真实任务。定义没有歧义、稳健的成功标准。认真设计评分器，并结合使用多种类型。确保问题对模型而言足够困难。持续迭代评测，提高其信噪比。务必阅读完整记录！

AI Agent 评测仍然是一个处于起步阶段、快速发展的领域。随着 Agent 承担更长的任务、在多 Agent 系统中协作，以及处理越来越主观的工作，我们也需要调整技术方法。随着理解不断深入，我们会继续分享最佳实践。

致谢

本文由 Mikaela Grace、Jeremy Hadfield、Rodrigo Olivares 和 Jiri De Jonghe 撰写。我们还感谢 David Hershey、Gian Segato、Mike Merrill、Alex Shaw、Nicholas Carlini、Ethan Dixon、Pedram Navid、Jake Eaton、Alyssa Baum、Lina Tawfik、Karen Zhou、Alexander Bricken、Sam Kennedy、Robert Ying 及其他人的贡献。特别感谢在评测合作中让我们受益的客户与合作伙伴，包括 iGent、Cognition、Bolt、Sierra、Vals.ai、Macroscopic、PromptLayer、Stripe、Shopify、Terminal Bench 团队等。这项工作体现了多个团队的共同努力，他们帮助推动了 Anthropic 评测实践的发展。

附录：评测框架

一些开源和商业框架可以帮助团队实现 Agent 评测，无需从零搭建基础设施。如何选择，取决于 Agent 类型、现有技术栈，以及你需要离线评测、生产可观测性，还是两者兼备。

Harbor 专为在容器化环境中运行 Agent 而设计，提供了跨云服务商大规模运行试验的基础设施，以及定义任务和评分器的标准化格式。Terminal-Bench 2.0 等热门基准通过 Harbor registry 分发，因此很容易将成熟基准与自定义评测套件一起运行。

Braintrust 是一个将离线评测、生产可观测性与实验跟踪结合起来的平台，适合既需要在开发过程中迭代、又需要在生产环境中监控质量的团队。它的 `autoevals` 库包含针对事实性、相关性等常见维度的预置评分器。

LangSmith 提供执行轨迹追踪、离线与在线评测，以及数据集管理，并与 LangChain 生态紧密集成。

Langfuse 提供类似能力，是一种可自行托管的开源替代方案，适用于有数据驻留要求的团队。

Arize 提供 Phoenix，这是一个用于 LLM 执行轨迹追踪、调试，以及离线或在线评测的开源平台；它还提供 AX，一项在 Phoenix 基础上扩展了规模化、优化和监控能力的 SaaS 服务。

许多团队会组合使用多种工具，自建评测框架，或者仅以简单的评测脚本作为起点。我们的体会是：框架虽然能够有效加速进展、促进标准化，但它们的效果终究取决于其中运行的评测任务。通常，最好的做法是迅速选定一个适合工作流程的框架，再把精力投入评测本身，不断打磨高质量的测试用例和评分器。